SAMSUNG
SW
AI ACADEMY
FOR
YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

DataBase

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. DDL
2. VIEW
3. 조인(JOIN)

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DDL

DDL

Data Definition Language

- 데이터 정의어
- 테이블이나 인덱스, 뷰 등과 같은 데이터베이스 객체들을 생성, 삭제, 변경하기 위한 명령어

명령어	의 미
CREATE 데이터베이스객체 ~	데이터베이스 객체 생성
ALTER 데이터베이스객체 ~	데이터베이스 객체 수정
DROP 데이터베이스객체 ~	데이터베이스 객체 삭제

DDL

DATABASE 생성

- 테이블이나 인덱스, 뷰 등과 같은 데이터베이스 객체들을 생성, 삭제, 변경하기 위한 명령어

```
CREATE DATABASE 데이터베이스명
[ DEFAULT CHARACTER SET characterset명 ] [ COLLATE collation값 ];
```

```
SHOW VARIABLES LIKE 'character_set_database';
```

```
SHOW VARIABLES LIKE 'collation_database';
```

DATABASE 변경

```
ALTER DATABASE 데이터베이스명
[ DEFAULT CHARACTER SET characterset명 ] [ COLLATE collation값 ];
```

DDL

- DATABASE 삭제

```
DROP DATABASE 데이터베이스명
```

- DATABASE 사용

```
USE 데이터베이스명
```

DDL

TABLE 생성 - 1

- 테이블을 구성하는 컬럼의 데이터 타입과 무결성 제약조건을 정의함으로써 테이블의 구조를 정의
- https://dev.mysql.com/doc/refman/8.4/en/create-table.html

```
CREATE TABLE 테이블명
(
  컬럼명  데이터타입 [ DEFAULT  기본값 ] [ 컬럼 제약조건 ]
  [ , … ]
  [, 테이블 제약조건, … ]
)
```

- 테이블 이름 정의 방법
  - 문자(A-Z, a-z)로 시작, 30자 이내
  - 문자(a-z, A-Z), 숫자(0-9), 특수문자($, _, #) 사용 가능
  - 동일 사용자가 소유한 다른 객체의 이름과 중복 불가
  - 서로 다른 테이블에서 동일한 데이터를 저장하는 컬럼 이름은 가능하면 같은 이름을 사용

DDL

제약 조건

- 데이터의 무결성(Integrity)을 지켜주는 중요한 역할
- 테이블에 행이 입력, 갱신, 삭제 될 때마다 지정된 제약 조건을 반드시 만족해야 함
- 제약조건을 활성화, 비활성화 할 수 있는 융통성을 지님
- 테이블 생성 시나 생성 후 ALTER문으로 제약조건 추가 가능
- 컬럼 레벨 또는 테이블 레벨 제약조건 설정 가능

제약 조건	의 미
NOT NULL	컬럼 값으로 NULL을 허용하지 없음, 값이 반드시 필요함 컬럼 레벨만 가능
UNIQUE	컬럼의 중복 값을 허용하지 않음
PRIMARY KEY	컬럼은 NOT NULL이어야 하며 UNQUE해야 함
FOREIGN KEY	참조하는 부모 테이블의 컬럼의 값만 허용
CHECK	컬럼 값으로 가능한 값 리스트나 범위에 해당하는 값만 허용

DDL

- TABLE 생성 SCRIPT 분석

```
CREATE TABLE emp (
   empno          int                  NOT NULL,
   ename          varchar(10),
   job            varchar(9),
   mgr            int,
   hireadte       datetime             DEFAULT   CURRENT_TIMESTAMP,
   sal            float(7, 2),
   comm           float(7, 2),
   deptno         int ,
   PRIMARY KEY    (empno),
   KEY FK_DEPTNO (deptno),
   CONSTRAINT   fk_deptno   FOREIGN KEY (deptno)   REFERENCES   dept (deptno)
) ENGINE = InnoDB   DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_0900_ai_ci ;
```

TABLE 생성 - 2

```
CREATE TABLE 테이블명 LIKE 기존테이블명;
```

- 기존 테이블을 토대로 하여 테이블의 구조를 복사한 새 테이블 생성
- 외래키 제약조건은 복사되지 않음

TABLE 생성 with 서브 쿼리

```
CREATE TABLE 테이블명 [ ( 컬럼명  데이터타입 [ DEFAULT  기본값 ]  [ , … ] ) ]
AS
    Sub Query
```

- 서브 쿼리 결과의 구조대로 테이블 생성 및 레코드 삽입이 동시에 이루어짐
- CREATE TABLE 명령문에서 지정한 컬럼 수와 데이터 타입과 반드시 일치해야 함
- 컬럼 이름을 명시하지 않을 경우 서브 쿼리 컬럼 이름(별칭)과 동일하게 만들어짐
- 제약조건 NOT NULL 복사됨
    - 기본 키, 외래 키, 고유 키, 체크와 같은 무결성 제약조건은 사용자의 재정의 필요
- DEFAULT 복사됨
- 자동 증가키는 복사되지 않음

DDL

- TABLE 삭제
  - 구조와 데이터 모두 삭제

```
DROP TABLE 테이블명;
```

  - 데이터만 삭제

```
TRUNCATE [TABLE] 테이블명;
```

DDL

TABLE 변경

- https://dev.mysql.com/doc/refman/8.4/en/alter-table.html
- 컬럼 추가

```
ALTER TABLE 테이블명 ADD [COLUMN] 컬럼명 데이터타입 [ 제약조건 ] [FIRST | AFTER 컬럼명] ;
```

- 컬럼 변경

```
ALTER TABLE 테이블명 MODIFY [COLUMN] 컬럼명 데이터타입 [ 제약조건 ] [FIRST | AFTER 컬럼명] ;
```

```
ALTER TABLE 테이블명 CHANGE [COLUMN] 컬럼명 새컬럼명 데이터타입 [ 제약조건 ] [FIRST | AFTER 컬럼명] ;
```

컬럼 이름을 함께 변경하며
컬럼 속성 변경

- 컬럼 이름 변경

```
ALTER TABLE 테이블명 RENAME COLUMN 컬럼명 TO 새컬럼명 ;
```

- 컬럼 삭제

```
ALTER TABLE 테이블명 DROP [COLUMN] 컬럼명 ;
```

- 테이블 이름 변경

```
ALTER TABLE 테이블명 RENAME [TO|AS] 새테이블명 ;
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

VIEW

VIEW

VIEW

- 테이블이나 또다른 뷰를 기반으로 한 논리적인 가상(Virtual)의 테이블
- 데이터는 갖고 있지 않으나 기반(Base) 테이블의 데이터를 보거나 변경할 수 있는 창(Window) 같은 역할
- Stored Query라고도 불리는 데이터베이스 객체
- 장점
  - 복잡한 쿼리문을 뷰로 저장하여 반복적으로 사용 가능(복잡한 조인, 서브 쿼리 등을 사용한 경우 효율적)
  - 뷰에 접근하는 사용자에게 미리 정의된 결과만을 보여줌(뷰에서 조회하는 레코드와 컬럼만 볼 수 있음)
  - 뷰에 포함되는 컬럼은 베이스 테이블에 영향을 주지 않고 다른 이름으로도 참조 가능
- 단점
  - 데이터 조작은 제한 사항이 많음

VIEW

VIEW의 유형

단순 뷰(Simple VIEW)	복합 뷰(Complex VIEW)
• 단일 테이블에서만 조회하며 데이터를 가공하지 않음 (Raw data 그대로) • 데이터 조작(DML) 가능 • 원본 테이블의 필수 제약조건 만족 시	• 두 개 이상의 테이블에서 조회 • 데이터를 가공함(중복 제거, 그룹화 등) • 데이터 조작(DML) 원칙적 불가 / 엄격한 제한

▪ 뷰에서의 데이터 조작 불가능한 대표적인 경우

- DISTINCT 로 중복이 제거된 경우
- SUM, AVG 등 집계 함수를 적용한 경우
- 산술식이나 함수를 사용해 계산된 파생 컬럼을 갱신하려는 경우
- 원본 테이블의 필수 컬럼이 뷰에 포함되지 않은 경우 INSERT 시도하는 경우

VIEW

VIEW 생성

```
CREATE [ OR REPLACE ] VIEW 뷰명 [ ( 별칭 [, 별칭, … ] ) ]
AS
    Sub Query
[ WITH CHECK OPTION ];
```

서브 쿼리의 조건절을 제약 사항으로 갖는 뷰 생성
(조건에 맞지 않는 삽입, 수정 체크)

VIEW 삭제

```
DROP VIEW 뷰명;
```

VIEW 수정

```
ALTER VIEW 뷰명 [ ( 별칭 [, 별칭, … ] ) ]
AS
   서브 쿼리
[ WITH CHECK OPTION ];
```

VIEW

VIEW 생성

- 부서별 월별 급여를 피봇 테이블 형태로 조회하는 뷰 생성 후 활용

```
CREATE OR REPLACE VIEW v_emp_dept_job_sal
AS
    SELECT  deptno,
            IF(job='CLERK', sal, null) CLERK,
            IF(job='MANAGER', sal, null) MANAGER,
            IF(job='PRESIDENT', sal, null) PRESIDENT,
            IF(job='ANALYST', sal, null) ANALYST,
            IF(job='SALESMAN', sal, null) SALESMAN
    FROM    emp;
```



```
SELECT      deptno,
            COUNT(CLERK) CLERK,
            COUNT(MANAGER) MANAGER,
            COUNT(PRESIDENT) PRESIDENT,
            COUNT(ANALYST) ANALYST,
            COUNT(SALESMAN) SALESMAN
FROM        v_emp_dept_job_sal
GROUP BY    deptno;
```

```
SELECT      deptno,
            AVG(CLERK) CLERK,
            AVG(MANAGER) MANAGER,
            AVG(PRESIDENT) PRESIDENT,
            AVG(ANALYST) ANALYST,
            AVG(SALESMAN) SALESMAN
FROM        v_emp_dept_job_sal
GROUP BY    deptno;
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

조인(Join)

조인

조인(JOIN)

사원의 이름과 부서 이름을 조회 하려면?

EMP

부서이름이 없다

EMPNO	ENAME	DEPTNO	JOB	MGR	HIREDATE	SAL	COMM
7369	SMITH	20	CLERK	7902	1998-12-17 00:00:00	4800.00	NULL
7499	ALLEN	30	SALESMAN	7698	1999-02-20 00:00:00	3600.00	300.00
7521	WARD	30	SALESMAN	7698	2020-02-22 00:00:00	1250.00	500.00
7566	JONES	20	MANAGER	7839	2020-04-02 00:00:00	2975.00	NULL
7654	MARTIN	30	SALESMAN	7698	2020-09-28 00:00:00	1250.00	1400.00
7698	BLAKE	30	MANAGER	7839	2021-05-01 00:00:00	2850.00	NULL
7782	CLARK	10	MANAGER	7839	2021-06-09 00:00:00	2450.00	NULL
7788	SCOTT	20	ANALYST	7566	2022-07-13 00:00:00	3000.00	NULL

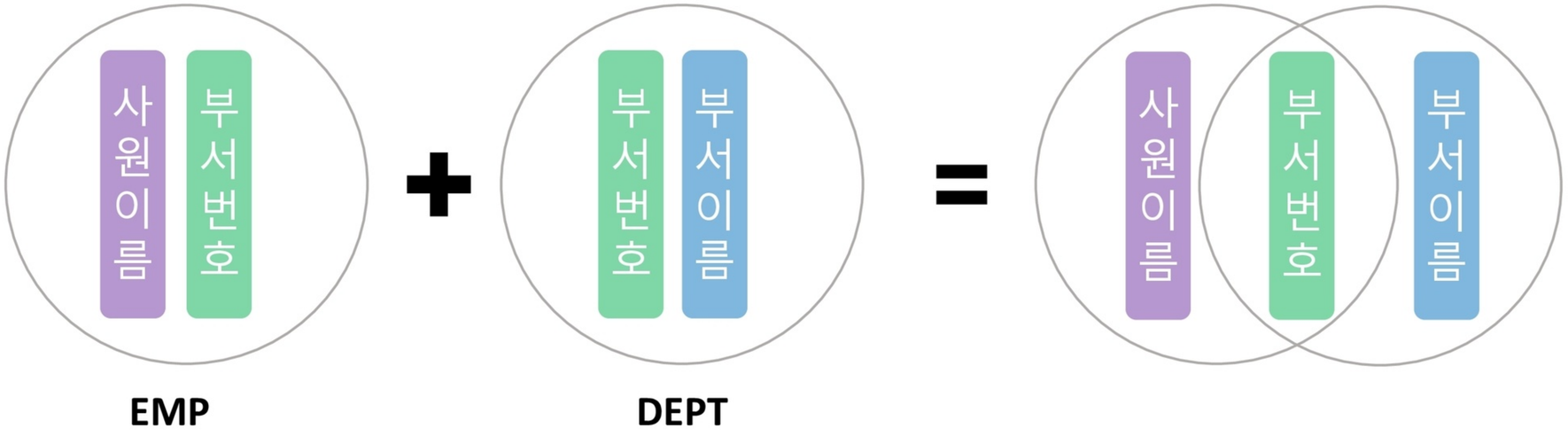
DEPT

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

두 테이블 연결 필요 → 조인 으로 해결

조인

- 조인(JOIN)
  - 하나의 SQL 명령문으로 여러 테이블에 저장된 데이터를 한번에 조회할 수 있는 기능
  - 두 개 이상의 테이블을 '결합' 한다는 의미
  - 주로 Primary-Key와 Foreign-Key의 관계를 가진 컬럼을 소유하고 있는 테이블을 통한 검색 시 사용
  - N개의 테이블 조인 시 N-1개 이상의 조인 조건 필요



조인

조인의 종류 - 1

조건에 따른 구분

- EQUI JOIN
  - ✓ 조인 조건의 컬럼을 '='(equal) 비교를 통해 같은 값을 가지는 행을 연결하여 결과를 생성하는 조인 방법
- NON-EQUI JOIN
  - ✓ <, BETWEEN a AND b 와 같이 '=' 조건이 아닌 연산자를 사용하여 조인하는 방법

조인 처리 결과에 따른 구분

- INNER JOIN
  - ✓ 조인 조건에 부합하는 행들만 연결하여 결과를 생성하는 조인 방법
- OUTER JOIN
  - ✓ 조인 조건에 부합하지 않는 기준 테이블의 행들도 모두 포함하여 결과를 생성하는 조인 방법

조인

조인의 종류 - 2

조인 조건 생략 시 구분

- NATURAL JOIN
  - ✓ 대상 테이블의 모든 공통 컬럼을 조인 조건 컬럼으로 사용하여 연결하는 조인 방법
- CROSS JOIN
  - ✓ 대상 테이블의 모든 행들을 가능한 조합으로 연결하는 조인 방법
  - ✓ TABLE1 8개 행
  - ✓ TABLE2 4개 행

32개의 행 결과 생성

그 밖의 조인

- SELF JOIN
  - ✓ 계층형의 성질을 갖는 데이터를 갖는 테이블에서의 상/하 관계의 행을 연결하여 결과를 생성하는 조인 방법

조인

EQUI JOIN

- 가장 많이 사용하는 조인 조건의 형태
- 조인 조건의 컬럼을 '='(equal) 비교를 통해 같은 값을 가지는 행을 연결하여 결과를 생성하는 조인 방법

```
SELECT      . . .
FROM        table1
JOIN        table2  USING(공통컬럼1 [, 공통컬럼2, ...] )
...
```

Using절 : 테이블명, 테이블 별칭 사용 불가!

```
SELECT      . . .
FROM        table1
JOIN        table2   ON(table1.col = table2.col  [ AND|OR  조건식2 ... ])
...
```

```
SELECT      . . .
FROM        table1, table2
WHERE       table1.col = table2.col
...
```

조인

EQUI JOIN

- 사원 이름과 부서 이름 조회

```
SELECT  ename, dname
FROM    emp
JOIN    dept  USING(deptno);
```

```
SELECT  ename, dname
FROM    emp
JOIN    dept   ON(emp.deptno = dept.deptno);
```

```
SELECT  ename, dname
FROM    emp,  dept
WHERE   emp.deptno = dept.deptno;
```

- 사원 이름과 부서 번호, 부서 이름 조회

```
SELECT  ename, dname, deptno
FROM    emp
JOIN    dept  USING(deptno);
```

```
SELECT  ename, dname, deptno  Column 'deptno' in field list is ambiguous
FROM    emp, dept
WHERE   emp.deptno = dept.deptno;  3ms
```

```
SELECT  ename, dname, dept.deptno
FROM    emp, dept
WHERE   emp.deptno = dept.deptno;
```

↓

```
SELECT  ename, dname, d.deptno
FROM    emp e, dept d
WHERE   e.deptno = d.deptno;
```

조인

NON-EQUI JOIN

- 조인 조건의 컬럼을 '='(equal) 비교가 아닌 다른 연산자(<, BETWEEN ~ AND , ...) 를 통해 행을 연결하여 결과를 생성하는 조인 방법
- JOIN ~ USING 사용 불가
- 사원의 이름, 월 급여, 급여 등급 조회

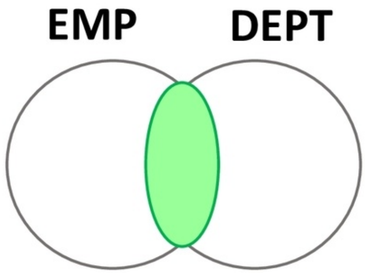
```
SELECT      e.ename, e.sal, s.grade
FROM        emp e
JOIN        salgrade s ON(e.sal BETWEEN s.losal AND s.hisal)
ORDER BY    3;
```

```
SELECT      e.ename, e.sal, s.grade
FROM        emp e, salgrade s
WHERE       e.sal BETWEEN s.losal AND s.hisal
ORDER BY    3;
```

조인

INNER JOIN

- 조인 조건에 부합하는 행에 대해서만 연결하는 조인 방식




```
SELECT              . . .
FROM                table1
[INNER] JOIN        table2      USING(공통컬럼1 [, 공통컬럼2, …])
…
```

```
SELECT              . . .
FROM                table1
[INNER]  JOIN       table2      ON(table1.col = table2.col  [ AND|OR  조건식2 … ])
…
```

```
SELECT      . . .
FROM        table1, table2
WHERE       table1.col = table2.col
…
```

조인

INNER JOIN

- sample data 추가 후 데이터 조회

```
INSERT INTO emp(empno, ename, sal, deptno, hiredate)
VALUES(9998,'TAEHEE',12000,20,now());
```

급여 범위 벗어남

조인 결과에서 누락

```
SELECT      e.ename, e.sal, s.grade
FROM        emp e
JOIN        salgrade s ON(e.sal BETWEEN s.losal AND s.hisal)
ORDER BY    3;
```

```
INSERT INTO emp(empno, ename, sal, hiredate)
VALUES(9999,'SSAFY',7000,now());
```

부서 배치 받지 못한 사원

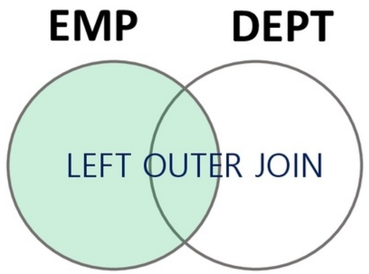
조인 결과에서 누락

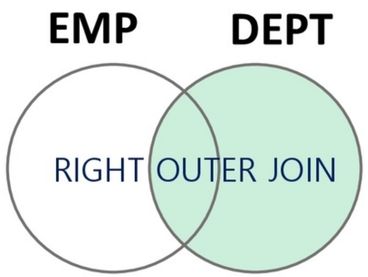
```
SELECT  ename, dname
FROM    emp
JOIN    dept  USING(deptno);
```

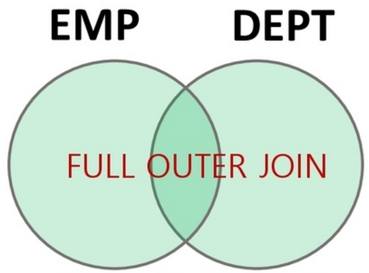
조인

OUTER JOIN

- 조인 조건에 부합하지 않는 행도 모두 포함하는 조인 방식
- 전체 행을 가져오고 싶은 테이블을 기준으로 함
  - LEFT OUTER JOIN : 기준 테이블을 먼저 기술
  - RIGHT OUTER JOIN : 기준 테이블을 나중에 기술
  - FULL OUTER JOIN : 양쪽 테이블 모두 기준 테이블







MYSQL 지원X

UNION으로 해결

```
SELECT                          . . .
FROM                            table1
LEFT|RIGHT [OUTER] JOIN         table2      USING(공통컬럼1 [, 공통컬럼2, ...])
...
```

```
SELECT                          . . .
FROM                            table1
LEFT|RIGHT [OUTER] JOIN         table2    ON(table1.col = table2.col  [ AND|OR  조건식2 ... ])
...
```

조인

OUTER JOIN

```
INSERT INTO emp(empno, ename, sal, deptno, hiredate)
VALUES(9998,'TAEHEE',12000,20,now());
```

급여 범위 벗어나는 사원

조인 결과에서 누락

```
SELECT      e.ename, e.sal, s.grade
FROM        emp e
JOIN        salgrade s ON(e.sal BETWEEN s.losal AND s.hisal)
ORDER BY    3;
```

```
INSERT INTO emp(empno, ename, sal, hiredate)
VALUES(9999,'SSAFY',7000,now());
```

부서 배치 받지 못한 사원

조인 결과에서 누락

```
SELECT  ename, dname
FROM    emp
JOIN    dept  USING(deptno);
```

조인

OUTER JOIN

- 급여 범위를 벗어나는 모든 사원 정보를 포함하여 조회하고 싶다면?

```
SELECT      e.ename, e.sal, s.grade
FROM        emp e
JOIN        salgrade s ON(e.sal BETWEEN s.losal AND s.hisal)
ORDER BY    3;
```

```
SELECT      e.ename, e.sal, s.grade
FROM        emp e
LEFT JOIN   salgrade s ON(e.sal BETWEEN s.losal AND s.hisal)
ORDER BY    3;
```

급여 범위 벗어나는 사원

조인 결과에서 누락

- 부서 배치 받지 못한 모든 사원 정보를 포함하여 조회하고 싶다면?

```
SELECT  ename, dname
FROM    emp
JOIN    dept  USING(deptno);
```

```
SELECT      ename, dname
FROM        emp
LEFT JOIN   dept  USING(deptno);
```

부서 배치 받지 못한 사원

조인 결과에서 누락

조인

- OUTER JOIN
  - 소속된 사원이 없는 모든 부서 정보를 포함하여 조회하고 싶다면?

```
SELECT              ename, dname
FROM                emp
RIGHT OUTER JOIN    dept USING(deptno);
```

조인

- NUTURAL JOIN
  - 조인 조건을 명시하지 않음
  - 조인 대상 테이블의 모든 공통 컬럼(이름이 같은 컬럼)을 이용하여 자연스럽게 조인
  - 주의 사항
    - 두 테이블에서 의미가 서로 다른 컬럼의 이름이 같을 경우 의도하지 않은 조인의 결과가 생성되므로 유의
    - 공통 컬럼이 없을 경우 CROSS JOIN과 결과 같음

```
SELECT          . . .
FROM            table1
NATURAL JOIN    table2
...
```

조인

- NUTURAL JOIN
  - 모든 사원의 이름, 부서 이름 조회

```
SELECT          ename, dname
FROM            emp
NATURAL JOIN    dept;
```

조인

CROSS JOIN

- 조인 조건을 명시하지 않음
- 두 조인 테이블을 조인할 조건이 없으므로 가능한 모든 연결을 다 시도하여 조인 결과 생성
- 두 테이블의 레코드가 많을 경우 너무 많은 결과 생성에 유의
  - 테이블1의 행 개수 * 테이블2의 행 개수의 결과만큼 생성

```
SELECT          . . .
FROM            table1
CROSS JOIN      table2
...
```

```
SELECT      . . .
FROM        table1, table2
...
```

조인

CROSS JOIN

- 사원과 부서의 가능한 모든 조합 행 생성

```
SELECT      ename, dname
FROM        emp
CROSS JOIN  dept;
```

```
SELECT      ename, dname
FROM        emp, dept;
```

조인

- SELF JOIN
  - 주로 계층형 테이블에서 나타나는 조인의 형태
  - 하나의 테이블을 마치 두 개의 테이블 처럼 조인
  - 상위 레코드와 하위 레코드를 연결하는 조인
  - 테이블 별칭 필수

EMP

EMPNO	ENAME	DEPTNO	JOB	MGR
7369	SMITH	20	CLERK	7902
7499	ALLEN	30	SALESMAN	7698
7521	WARD	30	SALESMAN	7698
7566	JONES	20	MANAGER	7839
7654	MARTIN	30	SALESMAN	7698
7698	BLAKE	30	MANAGER	7839
7782	CLARK	10	MANAGER	7839
7788	SCOTT	20	ANALYST	7566

EMP

EMPNO	ENAME	DEPTNO	JOB	MGR
7369	SMITH	20	CLERK	7902
7499	ALLEN	30	SALESMAN	7698
7521	WARD	30	SALESMAN	7698
7566	JONES	20	MANAGER	7839
7654	MARTIN	30	SALESMAN	7698
7698	BLAKE	30	MANAGER	7839
7782	CLARK	10	MANAGER	7839
7788	SCOTT	20	ANALYST	7566

```
SELECT    . . .
FROM      table1  a
JOIN      table2  b on (a.col = b.col)
…
```

```
SELECT      . . .
FROM        table1 a, table2 b
WHERE       a.col = b.col
…
```

조인

SELF JOIN

- 모든 사원과 자신의 상사(관리자) 이름 조회

```
SELECT  e.ename 사원이름 , m.ename 상사이름
FROM    emp e
JOIN    emp m ON(e.mgr = m.empno);
```

```
SELECT  e.ename 사원이름 , m.ename 상사이름
FROM    emp e, emp m
WHERE   e.mgr = m.empno;
```

- 상사(관리자)인 사원의 사번과 이름 조회

```
SELECT   distinct m.empno, m.ENAME
FROM     emp m
JOIN     emp e ON(m.empno = e.mgr);
```

```
SELECT  distinct m.empno, m.ENAME
FROM    emp m, emp e
WHERE   m.empno = e.mgr;
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Appendix

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

데이터 타입

데이터 타입

- MySQL의 데이터 타입
  - 숫자 데이터 형식
  - 문자 데이터 형식
  - 날짜 데이터 형식
  - 기타 데이터 형식

데이터 타입

숫자 데이터 타입

unsigned 지원 : 0 ~ 양수 2배

데이터 타입	크기	표현 범위	설명
BIT(비트 크기N)	1~64 비트	0 ~ 2^n-1	범위 내의 정수와 이진 표기의 문자열로 표현 ➔ (3을 표현하면 3 또는 b'11') 양수만 가능, 비트 연산 가능
TINYINT	1바이트	-128 ~ 127	정수
SMALLINT	2바이트	-32,768 ~ 32,767	정수
MEDIUMINT	3바이트	-8,388,608 ~ 8,388,607	정수
INT INTEGER	4바이트	-2,147,483,648 ~ 2,147,483,647	정수
BIGINT	8바이트	-9E-18 ~ 9E+18	정수
FLOAT	4바이트	-3.4E-38 ~ -3.4E+38	소수점 아래 7자리까지 표현
DOUBLE REAL	8바이트	-1.8E-308 ~ 1.8E+308	소수점 아래 15자리까지 표현
DECIMAL(p, [s]) NUMERIC(p, [s])	최대 29 바이트	-9,9E+34 ~ 9,9E+34	고정 소수점 표현(전체 자리수p, 소수이하자리수s) 최대 자릿수: 65 최대 소수점 이하 자리수: 30

데이터 타입

문자 데이터 타입-1

데이터 타입		최대 크기	설명
CHAR(n)		255개 문자	고정 길이 문자형(공백 패딩) CHAR(10) ➔ 문자 10개
VARCHAR(n)		65535개 문자	가변 길이 문자형(실제 사용하는 글자만큼의 공간만 사용) 실제 길이 저장하는 바이트 추가됨
BINARY(n)		255바이트	고정 길이 바이너리 데이터
VARBINARY(n)		255바이트	가변 길이 바이너리 데이터
TEXT	TINYTEXT	255바이트	대량 텍스트 저장 목적 길이 지정 불가능 인덱스의 제한적 사용
	TEXT	64KB	
	MEDIUMTEXT	16MB	
	LONGTEXT	4GB	
BLOB	TINYBLOB	255바이트	바이너리 데이터(이미지, 동영상 등) 저장 목적
	BLOB	64KB	
	MEDIUMBLOB	16MB	
	LONGBLOB	4GB	

데이터 타입

문자 데이터 타입-2

데이터 타입	설명
ENUM(값1,값2,...)	최대 65535개의 데이터 해당 데이터타입을 갖는 컬럼의 경우 명시된 데이터들 중 **하나**의 값만 가질 수 있음 단일 선택 ENUM('yes', 'no') 내부적으로 정수로 저장 ➔ yes: 1, no: 2
SET(값1,값2,...)	최대 64개의 중복되지 않는 데이터 해당 데이터타입을 갖는 컬럼의 경우 명시된 데이터들 중 **여러** 개의 값을 가질 수 있음 다중 선택 SET('apple', 'banana', 'orange') 내부적으로 비트마스크형태로 저장 ➔ apple&orange: 5

데이터 타입

날짜 데이터 타입

데이터 타입	크기(바이트 수)	설명
DATE	3	YYYY-MM-DD 형식으로 사용됨
TIME	3	HH:MM:SS 형식으로 사용
DATETIME	8	YYYY-MM-DD HH:MM:SS 형식으로 사용
TIMESTAMP	4	YYYY-MM-DD HH:MM:SS 형식으로 사용 time_zone 시스템 변수와 관련이 있으며 UTC 시간대로 변환하여 저장
YEAR	1	YYYY 형식으로 사용

협정 세계시(UTC)

1972년부터 시행된 국제 표준시

1970년 1월 1일 자정을 0 밀리초로 설정하여 기준을 삼아 그 후로 시간의 흐름을 밀리초로 계산

영국 : UTC+0
대한민국 : UTC+9

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

다음 방송에서 만나요!

삼성청년 SW·AI아카데미